Workshop on Usable Security and Privacy, Spring 2025 — *Mahmood Sharif*

Course Projects

Version No.: 1

1 Introduction

This document describes the projects proposed in this year's workshop. All projects share one common objective: ***Developing user-centered systems and tools to enhance user security and privacy***. As discussed in class, you should follow standard usable security and privacy practices to help accomplish this objective and evaluate the extent to which it is met. Among others, this includes the need to identify both the *usability* and the *security and privacy* metrics that the systems and tools aim to optimize, and design user studies for assessing them.

2 Important Notes

- You need to form teams consisting of **preferably three members** to work on the projects. All members are expected to contribute equally. Students should register their teams via Moodle.
- You are encouraged to select one of the projects proposed in this document. However, you also have the opportunity to pitch your own project ideas to the instructor for approval. You are required to update the Project Selection sheet on Moodle to indicate the selected project.
- Team registration and project selection should be completed by **May 11$^{\text{th}}$, 2026, 23:59**.
- All project descriptions contain references to related work. Please make sure to read the related work carefully before initiating the hands-on work. You may want to follow Keshav's advice on how to read academic research papers [7], especially if this is your first experience doing so.
- We recommend that you use a private GitHub repositories for version control and collaboration between the team members.
- It is allowed (even recommended) to leverage agentic AI development tools, such as Claude Code or OpenAI Codex, to assist in designing, implementing, and iterating on your projects. We ask that you document how such tools were used throughout your workflow, including decisions made, and any limitations encountered, as part of working on your project. You must also be prepared to justify and explain any design decision and implementation detail (i.e., you are accountable for your project, not your coding assistant!).

3 Presentations

Each team will present its project twice during the semester. The first, mid-semester presentation, will take place on **June 9$^{\text{th}}$, 2026**. In this presentation, you are expected to briefly describe the motivation of your project, prior work in the area, your current progress, and future plans to carry out the project. The second, (almost) final presentation, is scheduled for **July 14$^{\text{th}}$, 2026**. In this presentation, you should briefly remind your audience about the problem you want to solve, tell us

about the system or tool you developed, describe preliminary results, tell us about the planned user study, and finalize with remarks on what's remaining for wrapping up the project. Each team will be allocated ∼10 minutes for presenting its work, including Q&A. Both presentations will take place in the usual classroom (Kaplun 205), during the usual class time (16:00–18:00). The presentation slides should be uploaded to Moodle, preferably in a `.pdf`, `.pptx`, or `.keynote` format, prior to class.

4 Submission

The final submission consists of three primary parts: Code, a technical report, and a user-study protocol. If you elect to use coding agents, You are also encouraged to include Agents.md file (or similar) to demonstrate the gradual progression of your project.

4.1 Code

The code should contain your complete implementation. Additionally, you should include a `README.md` file (i.e., a Markdown file) specifying detailed instructions (commands, dependencies, ...) for how to execute your system or tool.

4.2 Technical Report

The technical report serves as a textual summary of your work throughout the course. We recommend that you structure it as follows:

1. **Abstract**: A concise summary of the report (∼200 words or fewer).
2. **Introduction**: Motivate the problem that your work seeks to address, and describe how it does so at a high level.
3. **Related work**: Describe prior efforts and where your work fits in (e.g., how it complements prior efforts).
4. **Methodology**: Detail and justify your technical approach, including the tools you used, the design choices you made, the metrics you used for evaluation, and the methods you used in your analyses. If you piloted your study, you should describe the user-study design, including, but not limited to, the study goals, the demographics of the pilot participants, and how they were recruited.
5. **Results**: Here you need to include the evaluation results (e.g., reporting on the appropriate metrics specified in the methodology section) to demonstrate the utility of the system or tool you developed.
6. **Discussion**: Here you should discuss the broader consequences of your project (e.g., on user security and privacy or future research), the potential limitations of the work, and potential future work and extensions you envision. If you have not piloted your study, then you should describe it as part of the future work.
7. **Conclusion**: Wrap up your work to leave the reader with the main contributions and takeaways.
8. **Contributions**: Shortly describe what were the contributions of each team member.
9. **LLM Statement**: A detailed statement about how you used LLMs and coding assistants as part of the project and while drafting the report. This should address both aspects that

worked well and ones that worked less well. Remember that you are have the full responsibility for the correctness and originality of your implementation and report.

10. **References**: The bibliography cited from within the main body of the report should be placed at the end.

The report should be **four to eight pages long, typeset in `LaTex`, and formatted using the ACM double-column conference template** (see instructions on Moodle). The final submission should contain both the source and the PDF.

4.3 User-Study Protocol

The user-study protocol should be submitted in a PDF titled `user-study-protocol.pdf`. You can find references to example protocols on Moodle.

4.4 How to Submit

You should submit a compressed tarball titles `group-<groupnumber>.tar.gz` (where `<groupnumber>` is a placeholder for your group's number), containing all three parts. To create the compressed tarball, first make a directory called `group-<groupnumber>/` containing: `code/` directory (including your implementation and `README.md`), `report/` (including both the PDF and source in LaTex), and `user-study-protocol.pdf`. Subsequently, execute the command `tar -czvf <groupname>.tar.gz <groupname>/`.

The final submissions should be uploaded to Moodle by **September 10st, 2026, 23:59**.

5 Project Descriptions

5.1 Implementing an Agentic Security Operation Center (SOC)

Analysts in security operation centers (SOCs) are continuously required to perform triage and respond to incidents, as they are constantly called to address alerts from various security tools, such as security information and event management (SIEM) and extended detection and response (XDR) [6, 17]. While these alerts sometimes correspond to true alarms, many correspond to false alarms. Furthermore, the risk scores assigned to alerts my not reflect their true severity, and alerts from different tools may present conflicting information. Altogether, this results in alert fatigue [22]. In this project, you will devise (semi-)autonomous solutions for aiding security analysts. To this end, you will implement, deploy, and evaluate AI agents that either autonomously respond to incidents, or guide analysts in incident response to help boost their productivity.

A scalable, event-driven architecture should ingest alerts from SIEM/XDR systems into a centralized pipeline (e.g., message queue or streaming layer), where they are normalized, enriched with contextual data (e.g., threat intel or asset data), and stored for processing. An agentic AI layer can be added to orchestrate multiple specialized agents (e.g., correlation agent, triage agent, and response agent) that analyze alerts, prioritize incidents, and either recommend or execute remediation actions, while maintaining state and audit logs. To empower analysts and facilitate their work, a user-facing interface and API layer should allow humans-in-the-loop (HITL) review decisions, override actions, and provide feedback, enabling continuous learning and improvement of the system.

Potential work plan

1. Familiarize yourself with related work [6, 17, 22] and popular systems that aid security analysts in incident response, such as AWS GuardDuty, Wazuh, Amazon Detective, Cloudwatch, and/or Security Hub.
2. Design an agentic system that simulates analysts with different roles in the SOC.
3. Deploy a single AWS host or cluster, install/enable monitoring tools.
4. Simulate reconnaissance techniques, attacks (e.g., single- or multi-host [2, 16]), misconfiguration and insecure installations, as well as benign usage, and explore the resulting true and false alarms, and potential false negatives.
5. Deploy your agentic SOC solution, through leveraging built-in incident response (sub-)agents (e.g., Inspector Agent, GuardDuty Agent, and/or System Manager Agent), and/or building your own (e.g., check out LangGraph or CrewAI) while providing it with appropriate tools (learn about MCP).
6. Study different trade-offs attainable with closed- or open-weights models.
7. Consider different visualizations methods for empowering HITL.

5.2 Designing a Malware Analyst (Sub-)Agent

Malware detection is a fundamental security problem, where the aim is to determine whether a program (e.g., a Windows binary, or so-called portable executable) is benign or malicious [8, 14]. When a previously unseen malicious program is detected, malware analysis experts spend a substantial amount of their time examining it to identify tell-tales of malice (e.g., malicious code) and discover potentially new attacker capabilities [3]. To this end, malware analysts often use binary-analysis tools (e.g., Ghidra or IDA Pro) to disassemble binaries (i.e., translating machine code to assembly) and read through the resulting assembly to find malicious blocks or functions. However, as the majority of code, even in malware, is not necessarily harmful, significant malware analyst effort and time is spent inspecting benign code.

In this project, your aim is develop a malware detection and analysis agent that can potentially help augment the agentic SOC. Besides detecting malicious programs, this agent can also help improve analyst productivity by offering means to point them toward malicious code. In particular, by explaining the decision of state-of-the-art machine-learning-based malware detectors [5, 8, 10] and integrating human-developed rules [11], the agent would identify and annotate instructions and/or features in the binary that make it more likely to be classified as malicious. The annotations would then enable malware analysts identify signs of malice more quickly and efficiently understand why certain programs were classified as malicious.

Potential work plan

1. Read related work on developing reliable malware detectors [8, 10, 14], and standard techniques for explaining the output of such detectors [8, 5, 19].
2. Familiarize yourself with relevant tools, including the Ghidra reverse-engineering tool and its plug-ins [13], the CAPA tool for identifying capabilities in binaries [11], and machine-learning model interpretability methods [21].
3. Train malware detectors, then implement explanation techniques, and execute the latter on malware detectors to identify bytes and features that most contribute to maliciousness.

4. Leverage Ghidra, possibly using a plugin, to map bytes and/or features that contribute to maliciousness so that they can be presented to analysts.
5. Provide the above as tools for (sub-)agents that can streamline the malware detection and analysis process, possibly summarizing explanations and analysis results in natural language.

5.3 Data-Privacy Inspector (Sub-)Agent

Organization-level privacy policies, regulations (e.g., HIPAA [23]), or user preferences [18, 20] may mandate certain policies for handling sensitive data. For instance, a system governed by HIPAA may allow storing patient identifiers (e.g., name, ID) in one database and clinical data (e.g., diagnoses, blood test results) in another, but prohibit storing both together in a single repository to reduce re-identification risk. The system would use a obfuscation and encryption to protect direct identifiers in the clinical database. Subsequently, access controls and audit logs ensure that only authorized workflows can temporarily join the data when needed, while all other processes operate on de-identified datasets. Naturally, potential access control misconfigurations may grant unauthorized parties and workflows access to sensitive data. In this project, you will implement an agentic system that detects potential privacy violations (e.g., by detecting sensitive data [12, 25] that is stored insecurely), alerts admins, suggests solutions, and, potentially, resolves violations autonomously. Such system would help streamline the privacy engineer role as part of larger agentic system (e.g., in agentic coding environments).

Potential work plan

1. Inform yourself about related work on privacy regulations [23], policies, and preferences [18, 20], and existing methods for detecting sensitive data [12, 25].
2. Implement a system/cluster hosting sensitive data, define appropriate policies, and simulate scenarios with or without violations. Such violations may be inherent to the system design or configuration, or may appear due to re-configuration or insertion/modification of data.
3. Develop and evaluate detection methods (e.g., [12, 25]). You may want to consider deploying Amazon Macie as part of your solution. Violations may need to be prioritized according to severity.
4. Implement real-time enforcement (blocking, redacting, or flagging violations), maintain audit logs, and allow centralized policy configuration aligned with regulations like HIPAA or GDPR, ensuring workflows or other sub-agents consistently comply with privacy policies throughout the development lifecycle. Design and implement automated remediation method using Agentic AI or otherwise. The system architecture would likely combine: A policy layer, a detection layer, and remidiation (possibly using dedicated MCP servers), as well as agent observability, auditing, and orchestration (e.g., LangChain/CrewAI).

5.4 Secure AI Agents via Human-AI Collaboration

AI-based Agentic systems are transforming the way we accomplish various tasks. For instance, as reflected from the projects described above, agentic systems are re-defining the way computer security and privacy problems are tackled. As another example, operating system (OS) agents (a.k.a. AI-based personal assistants or computer-use agents) enable new means for accomplishing various tasks on OSes: Instead of manually performing time-consuming tasks such as renaming files, exporting documents to PDF, or visiting different websites to download content, OS agents can now

perform these tasks automatically through simple instructions in natural language [4]. At the same time, however, adversaries can exploit weaknesses in agents to mislead them into taking harmful actions [1, 9]. For instance, an attacker posting an adversarial image on social media may mislead an OS agent into visiting a malicious website. To help mitigate these kinds of attacks, in this project, you will devise new methods for human-AI teaming (e.g., instructing humans to approve agents' actions when necessary) to ensure that the agents adhere to recommended security advice [15].

Potential work plan

1. Familiarize yourselves with OS agents [4] and attacks against them [1, 9], and read related work on recommended security advice [15].
2. Deploy OS agents using open- or closed-weights models and evaluate their utility on standard benchmarks (e.g., AgentDojo or WebArena).
3. Experiment with attacks on OS agents (ideally, within a virtual machine), and think about ways they can be further extended (e.g., by achieving other goals or exploiting other attack vectors). Note that attacks may also target any of the system described in the projects above.
4. Consider extending agent deployments with technical defenses (e.g., AGrail or Athena [24]) and evaluate how these impact attack success.
5. Develop interfaces to involve humans in the loop, and help defend against attacks and inform users about them, while still enabling agents to complete user-defined tasks.

References

[1] L. Aichberger, A. Paren, Y. Gal, P. Torr, and A. Bibi. Attacking multimodal os agents with malicious image patches. *arXiv preprint*, 2025.

[2] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu. ATLAS: A sequence-based learning approach for attack investigation. In *Proceedings of USENIX Security*, 2021.

[3] S. Aonzo, Y. Han, A. Mantovani, and D. Balzarotti. Humans vs. machines in malware classification. In *Proceedings of USENIX Security*, 2023.

[4] R. Bonatti, D. Zhao, F. Bonacci, D. Dupont, S. Abdali, Y. Li, Y. Lu, J. Wagle, K. Koishida, A. Bucker, et al. Windows agent arena: A benchmark for AI agents acting on your computer. `https://www.microsoft.com/applied-sciences/projects/windows-agent-arena`. Last accessed on 2025-03-25.

[5] L. Demetrio, B. Biggio, G. Lagorio, F. Roli, and A. Armando. Explaining vulnerabilities of deep learning to adversarial malware binaries. *arXiv preprint*, 2019.

[6] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates. NoDoze: Combatting threat alert fatigue with automated provenance triage. In *Proceedings of NDSS*, 2019.

[7] S. Keshav. How to read a paper. *ACM SIGCOMM Computer Communication Review*, 37(3):83–84, 2007.

[8] M. Krčál, O. Švec, M. Bálek, and O. Jašek. Deep convolutional malware classifiers can learn from raw executables and labels only. In *Proceedings of ICLRW*, 2018.

[9] A. Li, Y. Zhou, V. C. Raghuram, T. Goldstein, and M. Goldblum. Commercial LLM agents are already vulnerable to simple yet dangerous attacks. *arXiv preprint*, 2025.

[10] K. Lucas, W. Lin, L. Bauer, M. K. Reiter, and M. Sharif. Training robust ML-based raw-binary malware detectors in hours, not months. In *Proceedings of CCS*, 2024.

[11] Mandiant. CAPA: Extract capabilities from executable files. `https://mandiant.github.io/capa/`. Last accessed on 2025-03-25.

[12] Microsoft. Presidio: Data protection and de-identification SDK. `https://microsoft.github.io/presidio/`. Last accessed on 2025-03-25.

[13] National Security Agency. The ghidra software reverse-engineering framework. `https://github.com/NationalSecurityAgency/ghidra`. Last accessed on 2025-03-25.

[14] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas. Malware detection by eating a whole EXE. *arXiv preprint*, 2017.

[15] E. M. Redmiles, N. Warford, A. Jayanti, A. Koneru, S. Kross, M. Morales, R. Stevens, and M. L. Mazurek. A comprehensive quality evaluation of security and privacy advice on the web. In *Proceedings of USENIX Security*, 2020.

[16] A. Riddle, K. Westfall, and A. Bates. ATLASTv2: ATLAS attack engagements, version 2. *arXiv preprint*, 2023.

[17] M. Sharif, P. Datta, A. Riddle, K. Westfall, A. Bates, V. Ganti, M. Lentz, and D. Ott. DrSec: Flexible distributed representations for efficient endpoint security. In *Proceedings of S&P*, 2024.

[18] P. Shojaei, Y.-W. Chow, and E. Vlahu-Gjorgievska. User privacy concerns and preferences in mHealth applications: Balancing privacy and usability. In *Proceedings of MEDINFO*. 2025.

[19] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. In *Proceedings of ICML*, 2017.

[20] J. Tan, M. Sharif, S. Bhagavatula, M. Beckerle, M. L. Mazurek, and L. Bauer. Comparing hypothetical and realistic privacy valuations. In *Proceedings of WPES*, 2018.

[21] G. Tanner. Overview of different model interpretability libraries. `https://github.com/TannerGilbert/Model-Interpretation`. Last accessed on 2025-03-25.

[22] S. Tariq, M. Baruwal Chhetri, S. Nepal, and C. Paris. Alert fatigue in security operations centres: Research challenges and opportunities. *ACM Computing Surveys*, 57(9):1–38, 2025.

[23] US Gov. Health insurance portability and accountability act (hipaa). `https://www.hhs.gov/hipaa/index.html`. Last accessed on 2026-05-04.

[24] X. Wu, G. Hong, Y. Chen, M. Liu, F. Jin, X. Pan, J. Dai, and B. Liu. When bots take the bait: Exposing and mitigating the emerging social engineering attack in web automation agent. *arXiv preprint*, 2026.

[25] J. Zhou, E. Xu, Y. Wu, and T. Li. Rescriber: Smaller-LLM-powered user-led data minimization for navigating privacy trade-offs in LLM-based conversational agent. In *Proceedings of CHI*, 2025.